Java OOP Revision Notes

For 2+ year Java backend interviews and last-minute product-company revision

60-second recall

Concept	Simple meaning	How to say it in interview	Classic trap
Encapsulation	Hide data, expose controlled access	Private fields + meaningful methods	Getter returning mutable object
Inheritance	Reuse common code through IS-A relation	Child gets parent fields/methods	Using inheritance for HAS-A
Polymorphism	Same call, different behavior	Reference type vs object type	Expecting child-only methods on parent ref
Abstraction	Show what, hide how	Contract or shared base	Assuming interface and abstract class are the same

How to use this note

- Read the summary first, then revise one pillar at a time: encapsulation -> inheritance -> polymorphism -> abstraction.
- For each pillar, memorize: definition, when to use, one code pattern, and one interview trap.
- Before the interview, only reread the 'Last-minute traps' section and the boxed examples.

1) Encapsulation

- Meaning: keep the data private and let outside code interact through methods.
- Java idea: classes hide fields; methods act as the controlled entry point.
- Official Java docs describe encapsulation as hiding an object's data and methods from the rest of the world, with access restricted to public features and private internals. This matches the common private-field + getter/setter pattern.

What interviewers expect

- You should explain why private fields protect state.
- You should be able to show validation in constructor or setter.
- You should know that getters/setters are optional; meaningful domain methods are often better.
- You should know why returning a mutable object directly can break encapsulation.

Good pattern

```
class BankAccount {
    private int balance;

    BankAccount(int openingBalance) {
        if (openingBalance <= 0) throw new IllegalArgumentException("Invalid opening
balance");
        this.balance = openingBalance;
    }

    public String withdraw(int amount) {
        if (amount <= 0) return "Invalid amount";
        if (amount > balance) return "Insufficient balance";
        balance -= amount;
        return "Withdrawn. Balance = " + balance;
    }

    public int getBalance() {
        return balance;
    }
}
```

Encapsulation traps to remember

- Private does not mean immutable.
- final does not mean deep immutability.
- A getter that returns a mutable List, Date, or custom object can leak internal state.
- Business rules are often better inside methods like deposit(), withdraw(), approve(), not only setters.

2) Inheritance

- Meaning: a child class reuses fields and methods from a parent class.
- Use it only when the relationship is truly IS-A: Dog is an Animal, Manager is an Employee, Car is a Vehicle.
- Java supports single class inheritance: one direct superclass per class. Constructors are not inherited, but parent constructors can be invoked from the child.

- Private members are not directly accessible in the child. If a child needs data, use protected carefully or better, public methods/getters.

What interviewers expect

- Explain extends and the IS-A rule.
- Show constructor order: parent first, then child.
- Explain super for parent constructor, parent method, and parent field access.
- Explain field hiding versus method overriding.

```
class Vehicle {
    Vehicle() { System.out.println("Vehicle"); }
    void start() { System.out.println("Vehicle starting"); }
}

class Car extends Vehicle {
    Car() { System.out.println("Car"); }

    @Override
    void start() { System.out.println("Car starting"); }
}
```

Inheritance traps

- Car extends Engine is usually a bad design because a car HAS-A engine, it is not an engine.
- Fields are not polymorphic. If child and parent both declare the same field name, the reference type decides which one you see.
- Methods are polymorphic. Overridden methods are resolved at runtime.

3) Polymorphism

- Meaning: the same method call can behave differently depending on the actual object.
- Compile-time polymorphism = overloading. Runtime polymorphism = overriding.
- Reference type controls what you can call. Object type controls which overridden method runs.
- Oracle's Java tutorials describe polymorphism as different objects responding differently to the same message.

Very important rule

- Animal a = new Dog(); is allowed because Dog is an Animal.
- Dog d = new Animal(); is not allowed because not every Animal is a Dog.
- e.writeCode() fails when e is declared as Employee, even if the object is actually a Developer, because the compiler checks the reference type first.

Mini examples

```
// Overloading
int add(int a, int b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }

// Overriding
class Animal { void sound() { System.out.println("Animal"); } }
class Dog extends Animal { @Override void sound() { System.out.println("Bark"); } }
```

Upcasting and downcasting

- Upcasting: Employee e = new Developer(); -> automatic, safe, and common.
- Downcasting: Developer d = (Developer) e; -> explicit, risky unless you check instanceof first.
- instanceof is your safety check before downcasting.

Polymorphism traps

- Variables are not polymorphic the way methods are.
- A parent reference cannot directly access child-only methods.
- Overloading is decided at compile time; overriding is decided at runtime.

4) Abstraction

- Meaning: expose what is important, hide the implementation details.
- Abstract class: a partial blueprint with shared state and shared code. It may contain abstract and concrete methods.
- Interface: a contract/capability. A class promises to provide the behavior declared by the interface.
- Oracle's Java tutorials say abstract classes can mix abstract and non-abstract methods, can have fields, and cannot be instantiated. Interfaces define method signatures and a class can implement multiple interfaces.

When to choose abstract class

- You want shared fields like name, salary, id, or common helper methods.
- The classes are closely related.
- You want protected/private concrete methods or non-static state.

When to choose interface

- You want a pure contract like Payment, Flyable, Comparable, Runnable, Repository style roles.
- The classes are unrelated but share behavior.
- You need multiple inheritance of type.

```
abstract class Employee {
    private int workingDaysInMonth;

    Employee(int workingDaysInMonth) {
        this.workingDaysInMonth = workingDaysInMonth;
    }

    abstract void work();

    void login() {
        System.out.println("Logged in");
    }
}

interface PaymentService {
    void pay();
}
```

Abstract class vs interface in one line

- Abstract class = what you are (shared base + state).
- Interface = what you can do (contract / capability).
- Use interface for behavior variation; use abstract class when you need common state or shared implementation.

5) Last-minute traps interviewers love

- final on a variable means one assignment only. It does not make a mutable object deeply immutable.
- private hides access from other classes, but it does not prevent reassignment inside the same class.
- An interface can have many abstract methods. A functional interface must have exactly one abstract method.
- An abstract class can have a constructor. The constructor runs when a child object is created.
- A class can implement multiple interfaces, but it can extend only one class.
- Default methods in interfaces can create conflicts; the child class must resolve them.
- A class body is for declarations. Executable statements belong in methods, constructors, or initializer blocks.

Fast interview answers

Encapsulation: Hide data, expose safe behavior.

Inheritance: Reuse and extend a parent when the relation is IS-A.

Polymorphism: Same method call, different behavior based on object.

Abstraction: Show only what is necessary, hide how it works.

Interface: A contract/capability.

Abstract class: A partially complete base class with shared code or state.

6) Best revision order before a 2+ year interview

1. Read the 60-second summary.
2. Revise encapsulation with one bank-account example.
3. Revise inheritance with one parent-child example and one wrong HAS-A example.
4. Revise polymorphism with reference type vs object type.
5. Revise abstract class vs interface with one example each.
6. Do one final run of the traps section.

7) 10-minute self-test

7. Explain why Employee e = new Developer(); works but Developer d = new Employee(); does not.
8. Explain why e.writeCode() fails when e is declared as Employee.
9. Explain why a getter that returns List<String> can break encapsulation.
10. Explain why abstract class is better than interface for shared state.
11. Explain the difference between overloading and overriding.

Sources consulted while preparing this note

- Oracle Java Tutorials: Object-Oriented Programming Concepts
- Oracle Java Tutorials: Inheritance
- Oracle Java Tutorials: Polymorphism
- Oracle Java Tutorials: Abstract Methods and Classes
- Oracle Java Tutorials: What Is an Interface?
- dev.java: Introduction to Modules in Java